MERN STACK INTERNSHIP – DAY 3 ASSIGNMENT REPORT

Company

Insta Dot Analytics

Submitted By

Name: Chayan Soni

Roll No: PU02325EUGBTCS086

Course: B.Tech (CSIT), First Year

College: Symbiosis University of Applied Sciences

Submission Date: 03-06-2026

FOR SOURCE CODE:

GIT HUB :- https://github.com/chayan-soni/MERN-Internship

Objective

The objective of Day 3 was to integrate MongoDB with an Express.js application using Mongoose, replace file-based data storage with database operations, and implement centralized error handling middleware for consistent API responses and exception management.

Topics Covered

1. MongoDB and Mongoose Integration

- Connected the Express server to MongoDB using Mongoose.
- Created a structured User schema.
- Applied schema validation rules.

- Performed database operations using Mongoose models.
- Replaced mock/file-based data storage with MongoDB collections.

Concepts Learned

- MongoDB Collections and Documents
- Mongoose Schemas
- Models
- Validation
- Database Connectivity

2. Express Middleware Framework

- Used Express middleware to process incoming requests.
- Implemented JSON parsing middleware.
- Organized application routes and middleware.

Concepts Learned

- Middleware Functions
- Request Processing
- Route Handling
- Express Application Flow

3. Error Handling Middleware

- Developed centralized error handling middleware.
- Captured application errors using try-catch blocks.
- Returned structured JSON error responses.
- Improved API reliability and debugging.

Concepts Learned

- Error Middleware
- next() Function
- Exception Handling
- HTTP Status Codes

Project Structure

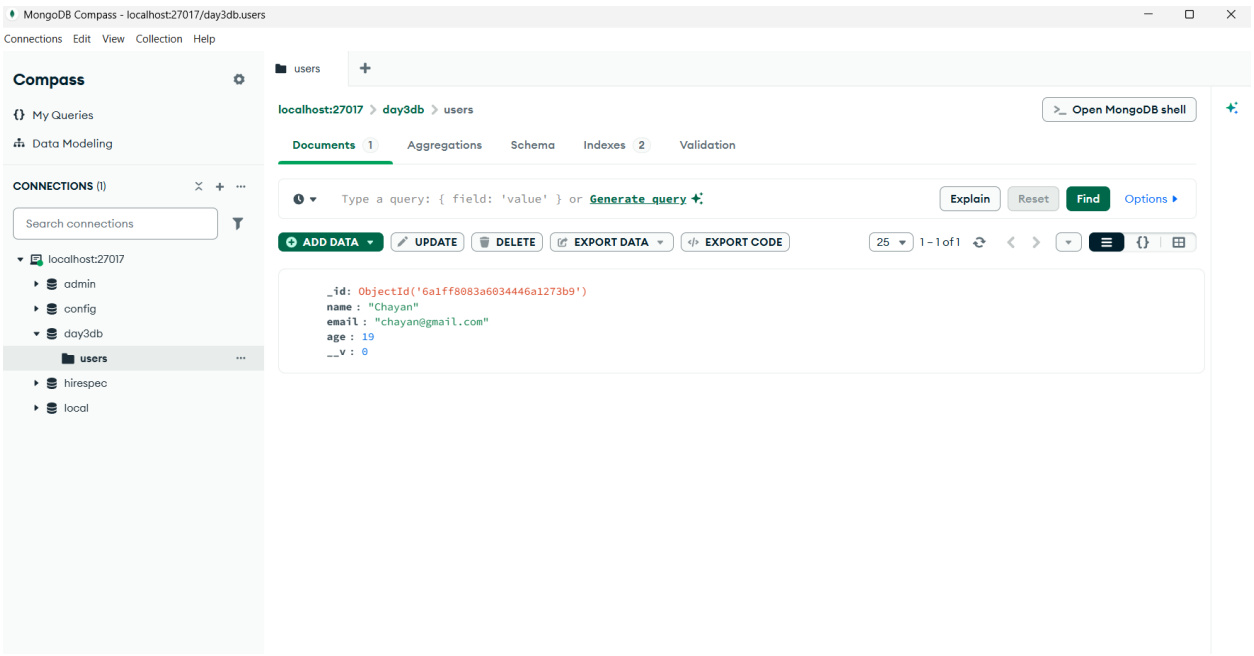
DAY 3

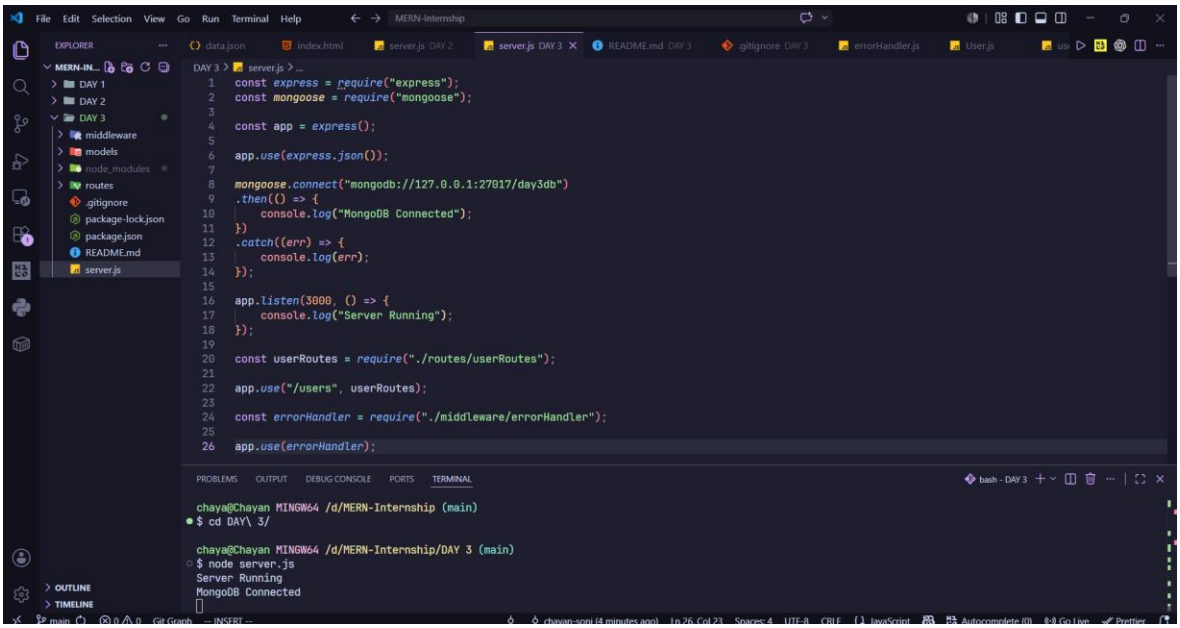
```
├── models
│   └── User.js
│
├── routes
│   └── userRoutes.js
│
├── middleware
│   └── errorHandler.js
│
├── server.js
│
├── package.json
└── package-lock.json
```

Implementation

The application was connected to MongoDB using Mongoose.

Screenshot





User Schema Creation

A User schema was created containing the following fields:

- Name
- Email
- Age

Validation rules were added to ensure required fields are provided before data is stored.

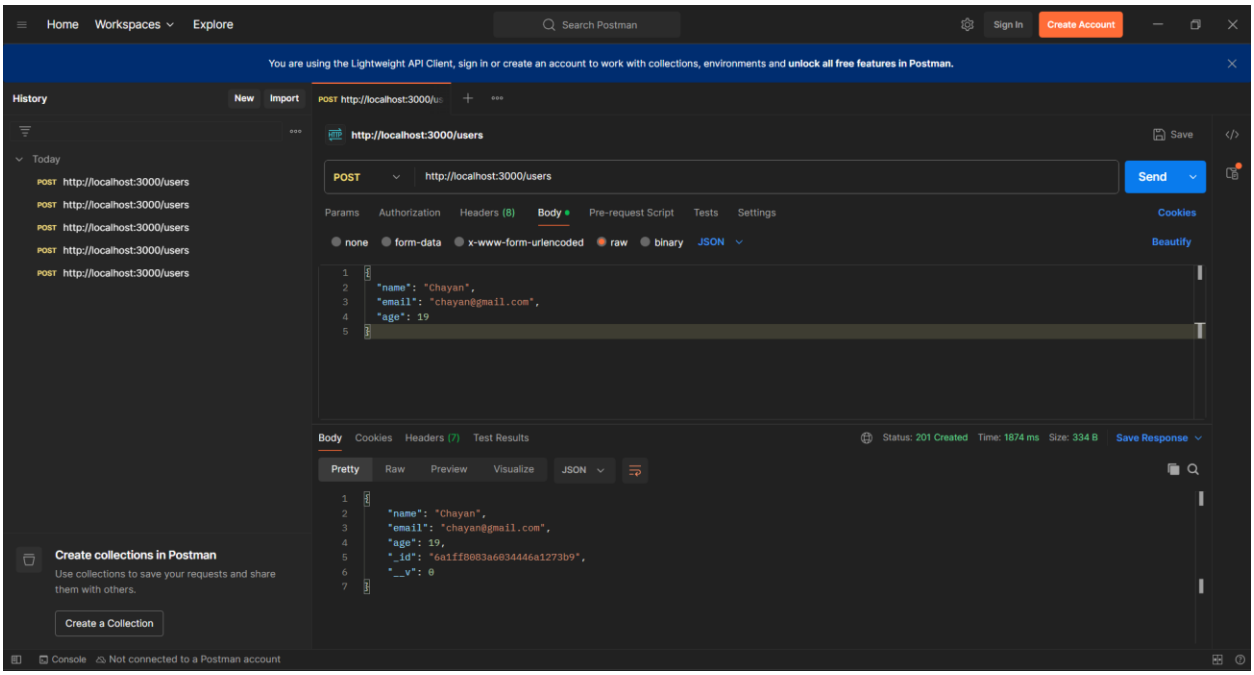
API Routes

The following API endpoints were implemented:

Create User

POST /users

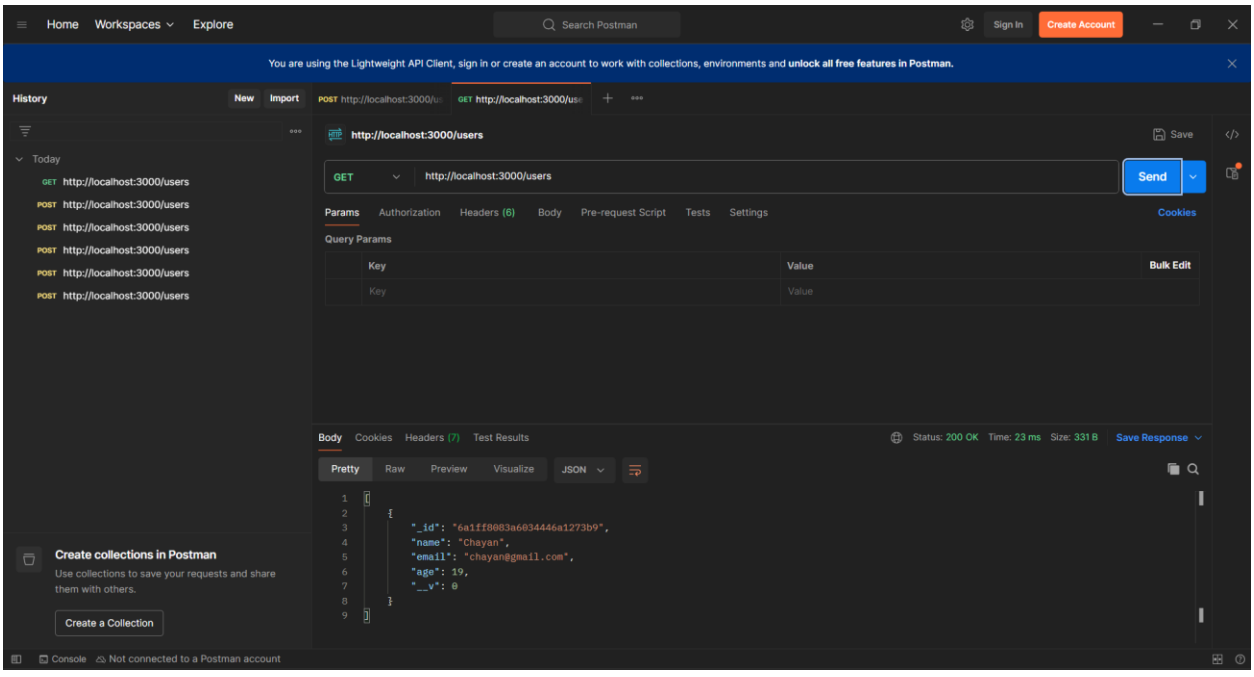
Used to create a new user record in MongoDB.



Get All Users

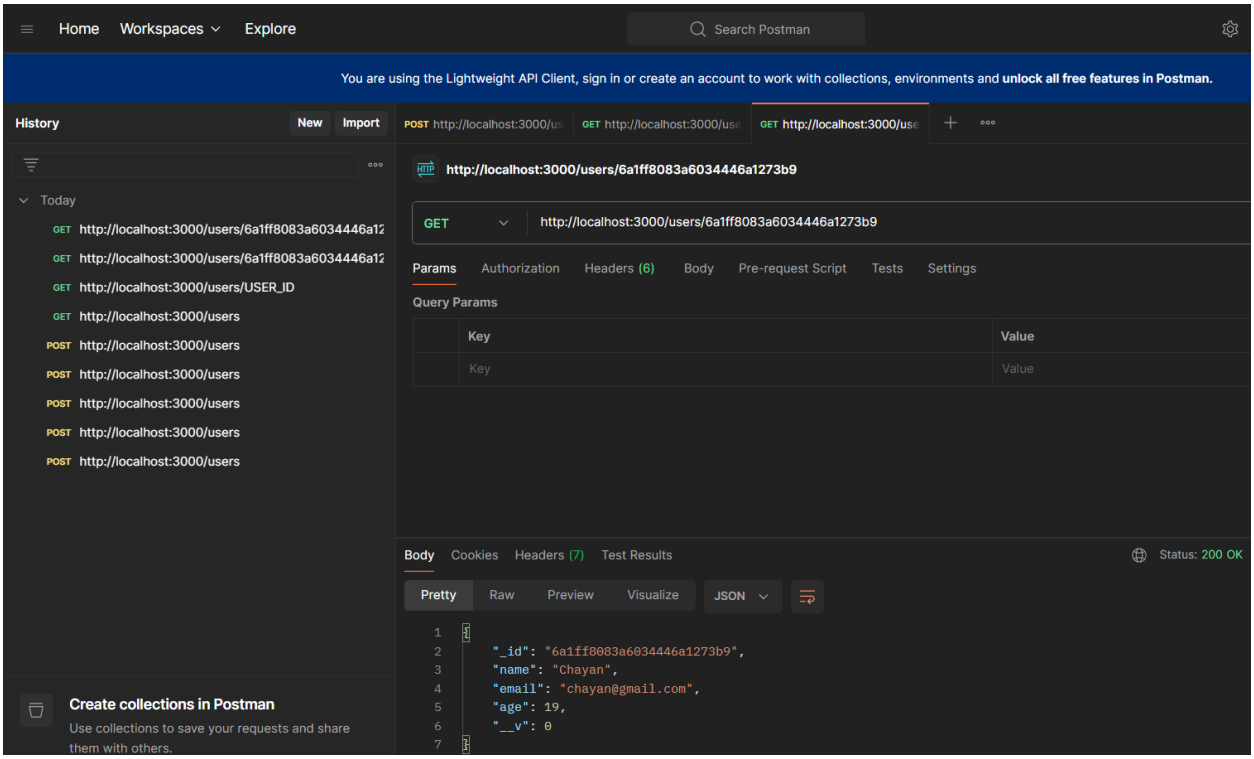
GET /users

Used to retrieve all users from the database.



Get User By ID

GET /users/ o



Error Handling Implementation

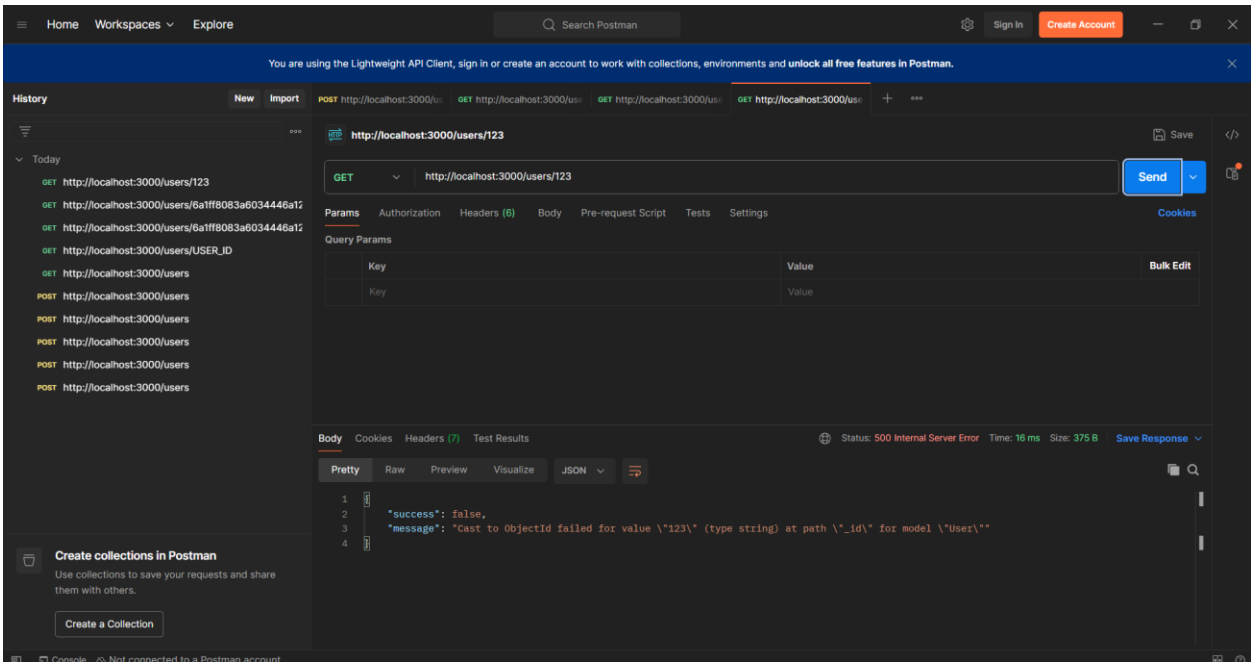
A centralized error handling middleware was implemented to handle:

- Missing required fields
- Invalid resource IDs
- Database validation errors
- Unexpected server exceptions

Sample Error Response

{
"success": false,
"message": "Validation error message"
}

Screenshot



Challenges Faced

- Installing and configuring required Node.js packages.
- Connecting Express to MongoDB using Mongoose.
- Understanding schema validation rules.
- Debugging request body issues in Postman.
- Implementing centralized error handling.

Learning Outcomes

By completing Day 3, I learned:

- How to connect Node.js applications to MongoDB.
- How to create schemas and models using Mongoose.
- How to perform CRUD-related database operations.

- How Express middleware works internally.
- How to implement centralized error handling.
- How to test REST APIs using Postman.

Conclusion

Day 3 focused on backend database integration and robust API error management. MongoDB and Mongoose were successfully integrated with the Express application, and centralized error handling middleware was implemented to improve reliability and maintainability of the application.